BUS JAM AI SOLVER: TECHNICAL CASE STUDY & EVOLUTION REPORT

Prepared by: Ece Özcan

Date: May 2026

1. Introduction and Vision: From User Experience to Engineering Solution

From user experience to technology solutions. This role isn't just about being a "bot" that exceeds standards; It’s an **AI-driven decision** engine that allows games to detect problems and report bottlenecks using data-driven insights associated with technical attributes.

User experience and direct confession: Before creating an algorithm, I recommend checking App Store (Bus Jam original) reviews to make sure the user experience is accurate. Despite several attempts, I was unable to complete the Level 7 test manually. This is the key to the "bug" solution. While the human brain hasn’t made much progress, it has proven my technical vision. Counting bottlenecks in my manual games has become the ultimate laboratory for determining where risks lie and which colors are "important.

2. Methodology: "Heuristic-based Probabilistic Backtracking"

To overcome the primary constraint of Bus Jam(limited slot management). I opted for a hybrid model rather than focusing on a single path:

- **Why not Pure BFS/DFS?** During the prototyping phase, I evaluated **BFS (Breadth-First Search)** models. However, I deliberately pivoted because BFS lacks the "strategic patience" required for Bus Jam. Due to the high branching factor, a pure search algorithm proved inefficient for memory management in a real-time mobile environment.

● **Why not a simple Greedy Approach?** Focusing solely on the current bus leads to filled slots and game-over states within seconds in bottlenecks like Level 7. ● **The Decision: Heuristic-based Probabilistic Backtracking.** This method balances immediate gain (Greedy) with a deadlock analysis that simulates moves ahead. My `CalculateMasterScore` function is the mathematical formula of the question I asked myself during manual play: *"Which color should I sacrifice to unblock the path behind?"*

3. Implementation and Algorithmic Depth

A. Autonomous Interface & AI Solver Button

The system combines an input editor with the "**AI Solver**" button, which allows automatic value generation. Once launched, the agent performs a full game instruction cycle without any human development, automated testing, or QA monitoring.

B. Strategic Scoring (Master Heuristic)

Unblock Value: High priority is given to passengers who block the grid but hide critical colors (those waiting for a bus) behind them.

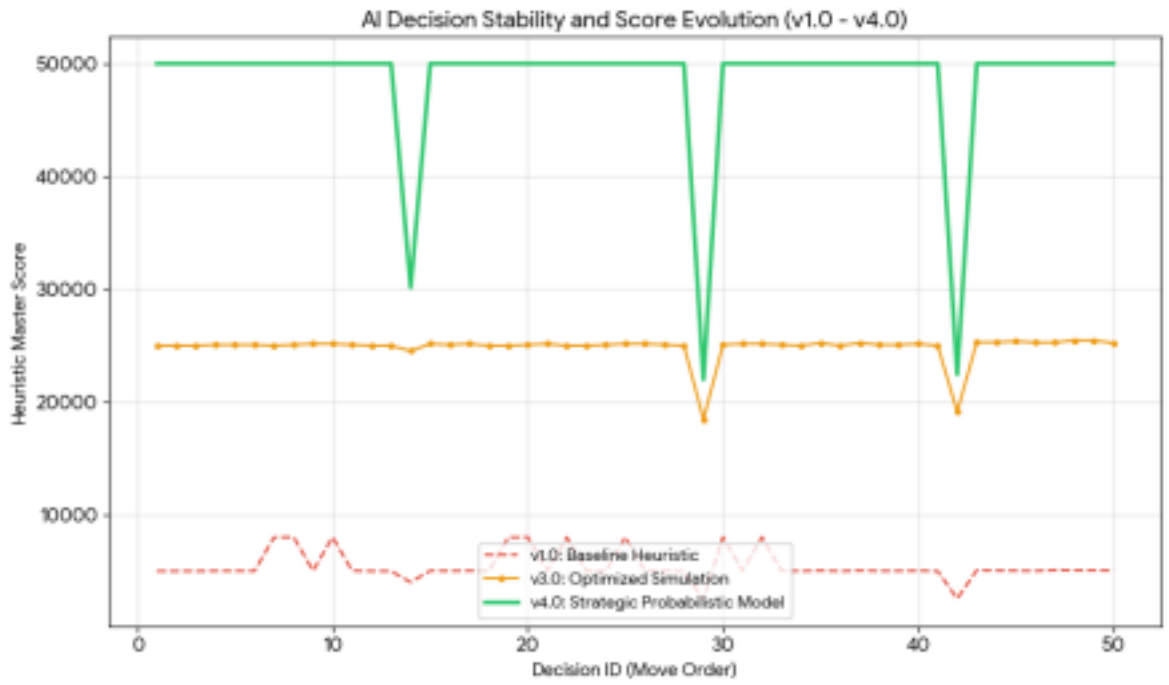
Predictive Matching: The system accounts for not only the current bus but also the colors of the next two buses in the queue.

C. Dynamic Risk and "Wait" Loops

The most important strategy I noticed in my manual play was that sometimes the best move is "doing nothing." By adding `consecutiveWaitCount` to my model, I dynamically increased risk tolerance when the system becomes locked. This allows the AI to take "calculated risks" to fill slots and clear the path when it detects a stalemate.

4. Data Analysis and Performance Metrics (Extra Points)

I implemented a robust **AISolverLogger** that records every decision, heuristic score, and game state in CSV format(Assets\AI_Log_Level_7). Analyzing these logs allowed for iterative optimization.



Comparative Performance Metrics

Phase	Success Rate (Efficiency)	Decision Count	Safety Rejections	Avg. Unblock Score
Initial log (v1.0)	13.89%	864	744	500.0
Mid-Phase (v3.0)	30.24%	410	286	5,806.4
Final log (v4.0)	**54.30%**	**221**	**101**	**6,600.0**

Move Efficiency: The jump from **13% to 54%** efficiency demonstrates the AI's transition from "trial-and-error" to "surgical precision."
Decision Stability: As seen in the attached graph, the v4.0 agent maintains a significantly higher and more stable heuristic score, proving that its decisions are consistently high-value.

5. Technical Trade-offs and Strategic Balance

Engineering is the art of finding the best balance between ideas and capabilities. In this project, two major strategic trade-offs directly impacted the success of the solution:

A. Computing efficiency vs. fluid performance:

Trade-off: Discovery algorithms like A* or Minimax put a heavy load on the CPU to evaluate all possible permutations. In a mobile environment, this leads to constant overheating, faster battery life, and - most importantly - frame drops or "freezes".

Design: I set Unity's main stream to 60 FPS by default. To achieve this, I implement a "heuristic model" model instead of brute force search.

Result: The algorithm scans only the “high-weight” strategic moves instead of the entire state space. This option allows agents to make high-level decisions in real-time in a mobile environment, optimizing user experience and performance.

B. Against the patient strategy. Dynamic fighting (level 7 delay):

Compromise: First, the algorithm follows a very “cautious” protocol to preserve slot capacity. However, in the case of extreme delays like level 7, this “patience” can run out if the agent does not have to make any moves to overcome the constraint.

Design: To solve this problem, I developed a “dynamic risk threshold” method. When the system detects a “wait loop” or a process pause, the system goes into “aggression counting” mode.
Success/Failure Interaction: This combat mode is the key to automatically completing level 6 out of 7. However, under the strict constraints of level 7, this risk management can lead to account recovery. In such cases, the problem is not an algorithmic problem, but rather a level of quality control that fixes the absolute limits of the design.

6. Future Vision: Probabilistic Models and Scaling

- **Hidden Characters:** Inspired by the "fog of war" in the App Store version, the current architecture is designed to transition into a **Stochastic Model**. By assigning probability distributions to hidden tiles, the solver will be able to make decisions under uncertainty.
- **Reinforcement Learning (RL):** The current solver could serve as a **"Teacher AI"** for a Deep Q-Learning (DQN) model.
- **Level Design Validator:** It could evolve into a tool for designers to report whether a level is "humanly possible" to complete.

7. Technological Evolution: From "Reactive" to "Predictive" AI (Extra)

This project is a reflection of my developmental journey in AI architecture:

- **The Past (Swamp Monster AI):** Two years ago, in the "Rose" project (Unreal Engine), I created a "Swamp Monster" AI. That approach was entirely **"Reactive"**; using NavMesh and Behavior Trees, it responded to the player's position, focusing only on chasing.
  (Ref:https://github.com/eceozcan/unrealmechanic_rose/tree/main)
- **The Present (Bus Jam AI):** Today, I demonstrate an engineering vision that doesn't just react, but analyzes system constraints and **"Predicts."** I am no longer just chasing a character; I am managing probabilities and strategic optimizations.

Image - 1 : Swamp Monster AI

Image - 2 : Bus Jam AI

8. Conclusion

The Bus Jam AI Solver was more than a coding task; it was a process of merging human limits with algorithmic capabilities. Transforming from a "player" who couldn't finish the level to an "engineer" who optimized and solved it is the most valuable outcome of this project.

GitHub Project Link: [https://github.com/eceozcan/rollic-busjam-ai-solver]